Lumio — Behind the Dashboard: the Machine-Learning Pipeline, Step by Step

Load → Clean → EDA → Feature engineering → Train/test split → Train → Evaluate → Tune → Score live data → Monitor

The live dashboard shows *results*: alerts, expected-vs-actual charts, the heatmap. This notebook opens the back end and runs **every step that produces those results, by hand, one cell at a time**.

This is not a copy of the back end. The notebook imports Lumio's own production modules (`lumio.ingest`, `lumio.quality`, `lumio.model`, `lumio.db`) and points them at a private notebook database, so what you see here is exactly what the app does.

Step	Pipeline stage	What you will see	Back-end code used
0	How live data flows	Does Lumio update with every transaction?	(explanation + diagram)
1	**Data load**	One transaction → 5-minute summary; load from **CSV** and from **SQL**; streaming ingestion	`world`, `db`, `ingest`
2	**Cleaning**	Validation, the data-quality gate (hold / dedup), aggregation	`quality.check`, `model.aggregate`
3	**EDA for modelling**	Targets, transformations, what drives them	—
4	**Feature engineering**	The 9 features Lumio uses and the ones it deliberately does *not* use	`model.features`, `world.is_payday`
5	**Train / validation / test split**	Time-based split, and why a random split cheats	—
6	**Model training & evaluation**	Seasonal baseline vs linear regression vs gradient boosting; MAE, RMSE, R^2, MAPE	same settings as `model.Baseline`
7	**Hyperparameter tuning**	Randomised search with time-series cross-validation; validation curve	—
8	**From prediction to anomaly score**	Noise model, robust per-segment spread, z-scores; proof the notebook = the app	`model.Baseline`
9	**Threshold tuning & detection metrics**	Precision / recall / F1 vs sensitivity; confirmation scans; incidents caught	same rules as `detector`
10	**Live scoring: what the dashboard shows**	Stream batches into the database, score them, rebuild the dashboard chart and heatmap from SQL	`ingest.store`, `bucket_scores` table
11	**Monitoring & retraining**	Daily accuracy, drift, saving and loading the model	`Baseline.save / load`
12	**Summary & professor's questions**	Pipeline map + likely viva questions	—

Run: from the `analysis` folder, `pip install -r requirements.txt` then *Run All* (about 2 minutes). It creates its own `pipeline_demo.db`; it never touches the live app's database.

---

0. How live data flows — *does Lumio update with every transaction?*

Short answer: every payment is counted, but not every payment triggers a calculation. Lumio works the way real monitoring works at a bank:

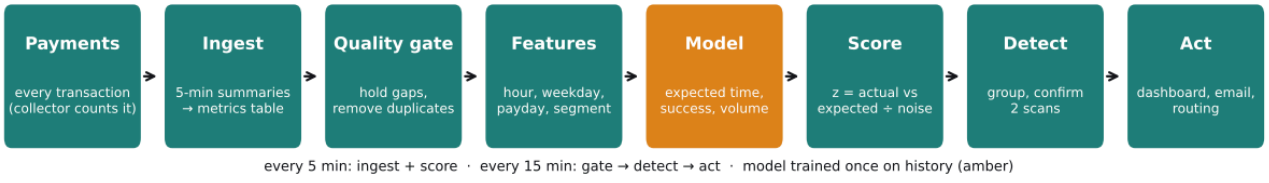
Layer	Happens	What it does
Each payment	Continuously (≈ 2 million a day in this network)	The bank's monitoring collector *counts* it: success or failure, how long it took, its amount, segment and app version
5-minute summary	Every 5 minutes	The collector sends **one row per segment** (48 rows): volume, successes, p95 time, average amount, plus a **small sample of individual transactions** for investigations
Ingest → database	Every 5 minutes	`ingest.store()` validates the batch and writes it to the `metrics` table
Scoring	Every 5 minutes	Each new row is compared with what the model *expects* for that segment at that moment → the live chart (`bucket_scores`)
Scan	Every 15 minutes	Quality gate → z-scores → grouping → 2-scan confirmation → alert, routing, email (`window_scores` , `incidents` , `outbox`)
Model training	**Once** on 3 weeks of history (in production: re-trained on a schedule, e.g. monthly)	Learns what *normal* looks like for every segment, hour, weekday and payday

So the **data** updates every 5 minutes, but the **model** does not change with each transaction, and it shouldn't:

1. **Stability:** if the model re-learned from every payment, an outage would slowly become the new "normal" and stop alerting. That is the classic *contamination* problem.
2. **Scale and cost:** summarising millions of payments into 48 rows per 5 minutes makes the maths cheap and fast.
3. **Statistics:** a single payment says nothing; a 5-minute summary of hundreds of payments has a measurable noise level (Step 8).
4. **Privacy:** summaries contain no card numbers or customer details (data minimisation).

In this demo the payments are simulated by `lumio/world.py` on top of real NAB demand data, and the dashboard's *Play* button replays them through exactly this flow at 1×–10× speed.

```
Lumio back end imported from: lumio-demo/lumio
Notebook database: pipeline_demo.db | live period: 2025-02-07 00:00:00 → 2025-02-
16 23:55:00
```



Step 1 — Data load

1.1 From one transaction to a 5-minute summary

What a collector sees is individual payments. Here we take one real segment-bucket from the dataset (*Bank X · UPI · Android · North, Fri 7 Feb 10:00*) and **simulate every individual payment** in it, then summarise them exactly as a collector does.

```
419 individual payments in 5 minutes for one segment. First 6:
```

	txn_id	time	amount_$	latency_ms	status
0	T100000	2025-02-07 10:00:00.824144	30.44	1217.9	success
1	T100001	2025-02-07 10:00:01.747379	21.65	1648.8	success
2	T100002	2025-02-07 10:00:02.127549	52.85	1270.9	success
3	T100003	2025-02-07 10:00:05.016846	49.03	601.7	success
4	T100004	2025-02-07 10:00:05.048185	26.01	1146.4	success
5	T100005	2025-02-07 10:00:05.950244	49.79	756.4	success

source	volume	success_count	p95_ms	avg_amount
collector summary from these payments	419	412	1924.6	36.80
row actually in the dataset	419	412	1877.6	37.58

419 transactions become 1 row. Every 5 minutes, 48 such rows arrive (one per segment), about 6,642 payments summarised per batch on average.

```
world.generate_bucket() → 48 summary rows + 284 sampled transactions + 7 service rows
```

	ts	bank	channel	platform	region	app_version	amount	merchant_category	latency_ms	status
0	2025-02-07 10:00:00	Bank X	UPI	Android	North	5.1	6.42	Grocery	823.1	success
1	2025-02-07 10:00:00	Bank X	UPI	Android	North	5.1	7.67	Fuel	1552.6	success
2	2025-02-07 10:00:00	Bank X	UPI	Android	North	5.1	31.31	Fuel	1216.9	success
3	2025-02-07 10:00:00	Bank X	UPI	Android	North	5.1	33.35	Bills	1326.4	success
4	2025-02-07 10:00:00	Bank X	UPI	Android	North	5.1	49.30	Bills	1147.3	success

```
The simulator's first summary row matches the CSV row: True
```

1.2 Load from CSV

```
CSV loaded in 0.3s: 290,304 rows × 10 columns, 76 MB in memory
```

	ts	bank	channel	platform	region	app_version	volume	success_count	p95_ms	avg_amount
dtype	datetime64[us]	str	str	str	str	float64	int64	int64	float64	float64

	ts	bank	channel	platform	region	app_version	volume	success_count	p95_ms	avg_amount
0	2025-01-17	Bank X	UPI	Android	North	5.1	290	286	1832.9	39.22
1	2025-01-17	Bank X	UPI	Android	South	5.1	265	257	1847.9	36.49
2	2025-01-17	Bank X	UPI	Android	West	5.1	190	184	1831.8	39.85

1.3 Load into SQL — Lumio's real database schema

`db.init_db()` creates the **same tables the live app uses** (SQLite here; PostgreSQL in Docker). The 3-week history goes in as a **bulk initial load** (batch ETL); the live days will arrive later as **5-minute streaming batches** (Step 10).

table	bucket_scores	cost_log	dq_checks	events	feedback	ground_truth	incidents	investigation_steps	maintenance
role	5-min expected vs actual → live chart		quality-gate decisions		acknowledge / dismiss	answer key (Proof tab)	alerts		

```
Bulk-loaded 290,304 metric rows + 42,336 service rows in 3.6s

SELECT ts, bank, channel, platform, region, app_version, volume, success_count,
p95_ms, avg_amount FROM metrics WHERE ts < '2025-02-07 00:00:00'
```

	CSV	SQL	match
rows	2.903040e+05	2.903040e+05	True
total payments	4.130342e+07	4.130342e+07	True
total successes	4.056504e+07	4.056504e+07	True
mean p95	2.354064e+03	2.354064e+03	True

	day	bank	payments	success_pct	weighted_p95_ms
0	2025-02-03	Bank W	277392	98.27	2471.0
1	2025-02-03	Bank X	556607	98.32	2082.0
2	2025-02-03	Bank Y	445689	98.29	2219.0
3	2025-02-03	Bank Z	333326	98.30	2342.0
4	2025-02-04	Bank W	311462	98.25	2507.0
5	2025-02-04	Bank X	626229	98.23	2122.0
6	2025-02-04	Bank Y	498425	98.27	2266.0
7	2025-02-04	Bank Z	375349	98.24	2388.0

1.4 Streaming ingestion: how live batches arrive

In the app, the simulator (or a real collector) posts a JSON batch every 5 minutes to `/api/ingest`, which calls `ingest.store()`. Here we call the same function for the first three live batches.

	batch time	metrics stored	services stored	samples stored
0	2025-02-07 00:00:00	48	7	0
1	2025-02-07 00:05:00	48	7	0
2	2025-02-07 00:10:00	48	7	0

```
Example of one JSON record as posted to /api/ingest:
{
 "ts": "2025-02-07T00:00:00",
 "bank": "Bank X",
 "channel": "UPI",
 "platform": "Android",
 "region": "North",
 "app_version": "5.1",
 "volume": 409,
 "success_count": 392,
 "p95_ms": 1774.8,
 "avg_amount": 37.54
}
```

Step 2 — Cleaning

2.1 Structural validation (at ingest)

`ingest.store()` rejects records missing a required field. Validation at the door is *structural only*; judging whether the data is **trustworthy** is the quality gate's job.

```
Records sent: 2  →  stored: 1 (the incomplete record was rejected)
```

	nulls	success_count > volume	volume ≤ 0	p95 ≤ 0	exact duplicates
rows failing check (history)	0	0	0	0	0

2.2 The data-quality gate (`quality.check`)

Every 15-minute scan, before anything is scored, each segment's data is checked for **completeness** (did all three 5-minute buckets arrive?) and **duplicates**. Two real problems are planted in the live period. Let's run the app's gate on them.

```
Normal window, 2025-02-07 09:00: 144 raw rows → 144 after de-duplication | status
counts: {'ok': 48}

Collector stopped (Bank Z West), 2025-02-07 14:00: 132 raw rows → 132 after de-
duplication | status counts: {'ok': 44, 'hold': 4}
```

	completeness	duplicates	status	note
Bank Z|UPI|Android|West	0.0	0	hold	only 0 of 3 buckets arrived — held, not scored
Bank Z|UPI|iOS|West	0.0	0	hold	only 0 of 3 buckets arrived — held, not scored
Bank Z|Card|Android|West	0.0	0	hold	only 0 of 3 buckets arrived — held, not scored
Bank Z|Card|iOS|West	0.0	0	hold	only 0 of 3 buckets arrived — held, not scored

```
Duplicate records (Bank W North), 2025-02-12 09:30: 156 raw rows → 144 after de-
duplication | status counts: {'ok': 44, 'dedup': 4}
```

	completeness	duplicates	status	note
Bank W|UPI|Android|North	1.0	3	dedup	3 duplicate records removed before scoring
Bank W|UPI|iOS|North	1.0	3	dedup	3 duplicate records removed before scoring
Bank W|Card|Android|North	1.0	3	dedup	3 duplicate records removed before scoring
Bank W|Card|iOS|North	1.0	3	dedup	3 duplicate records removed before scoring

Missing data is not a slowdown, and duplicated data is not a surge. Held segments are not scored (no false page); duplicates are removed before scoring. In the app these decisions go to the `dq_checks` table and the *Data quality* tab.

2.3 Aggregation to modelling windows (`model.aggregate`)

The model learns from **15-minute windows** (the scan interval): 3 × 5-minute buckets per segment, with app-version rows combined.

A subtle point: you cannot average two p95 values. During a release, one segment reports two rows (v5.1 fast, v5.2 slow). The p95 of all payments together is the 95th percentile of the **mixture** of both groups, which `mix_p95` solves numerically.

method	p95 ms
volume-weighted average of p95s (wrong)	3550
mix_p95 (Lumio)	5426
check: simulate 1,000,000 payments and take the 95th percentile	5426

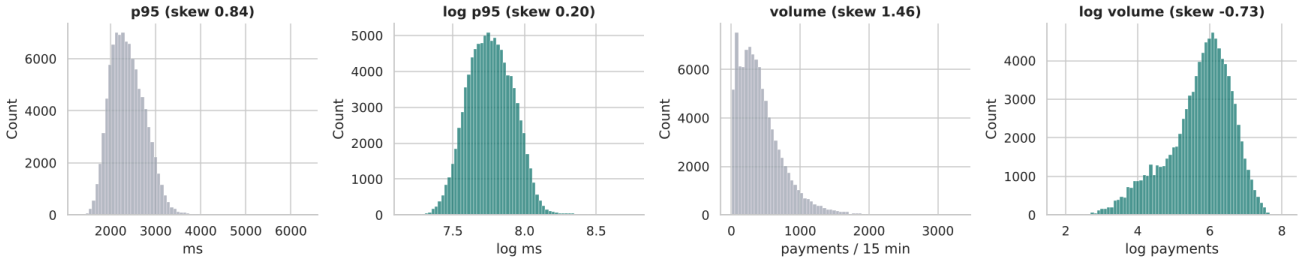
```
290,304 5-minute rows → 96,768 15-minute windows (48 segments × 2016 windows) in 0.6s
```

	ts	bank	channel	platform	region	volume	success_count	avg_amount	p95_ms	success	s
0	2025-01-17	Bank X	UPI	Android	North	825	814	39.406667	1821.852615	0.986667	Ba X|UPI|Android|No
1	2025-01-17	Bank X	UPI	Android	South	785	772	37.696667	1863.358004	0.983439	Ba X|UPI|Android|Sou
2	2025-01-17	Bank X	UPI	Android	West	516	505	38.126667	2084.489396	0.978682	Ba X|UPI|Android|We

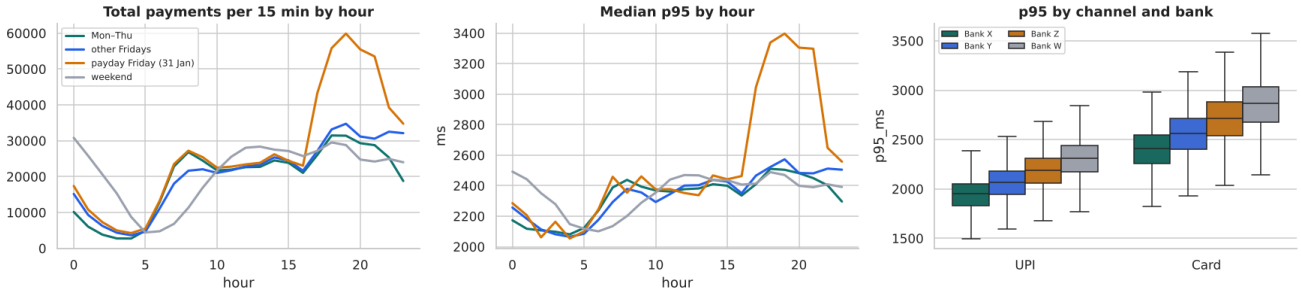
---

Step 3 — EDA for modelling

Three things the model must predict for every segment and moment: **payment time (p95)**, **success rate** and **volume**. EDA tells us how to treat each one.



Both p95 and volume are **right-skewed and multiplicative** (a slowdown is '×2', not '+500 ms'), so Lumio models their **logarithms**. Success rate is bounded between 0 and 1 and modelled directly, weighted by volume.



Volume and payment time both follow **hour of day**, differ on **weekends**, and the payday Friday evening carries **1.7×** the volume of a normal Friday evening. Channel and bank shift the whole level. These observations become the features in Step 4.

Step 4 — Feature engineering (`model.features`)

Feature	Type	Why
`hour`	numeric, **fractional** (10:15 → 10.25)	Time of day drives both demand and speed; fractional keeps 15-min resolution
`dow`	0–6	Weekly pattern
`weekend`	0/1	Weekend days behave differently (late nights, quiet mornings)
`payday`	0/1	15th / last day of the month, moved to Friday if on a weekend (`world.is_payday`)
`payday_eve`	0/1	Payday 16:30–22:30, when the rush happens
`bank_i` , `channel_i` , `platform_i` , `region_i`	categorical codes	Each segment has its own level; gradient boosting handles categories natively (no one-hot needed)

What is deliberately *not* a feature: anything measured *now*: current volume, error rate, upstream service time. The model's job is to say what **should** happen, using only things known in advance (the calendar and the segment). If it used the current error rate, it would "explain away" an outage as normal. This is a form of **target leakage**, and avoiding it is also why the app can pre-compute every expectation for the whole replay up front.

	ts	bank	channel	platform	region	hour	dow	weekend	payday	payday_eve	bank_i	channel_i	platform_i	regio
0	2025-01-31 18:30	Bank X	UPI	Android	North	18.50	4	0	1	1	0	0	0	
1	2025-01-31 10:15	Bank X	UPI	Android	South	10.25	4	0	1	0	0	0	0	
2	2025-02-01 02:00	Bank X	UPI	Android	West	2.00	5	1	0	0	0	0	0	
3	2025-02-03 09:45	Bank X	UPI	iOS	North	9.75	0	0	0	0	0	0	1	

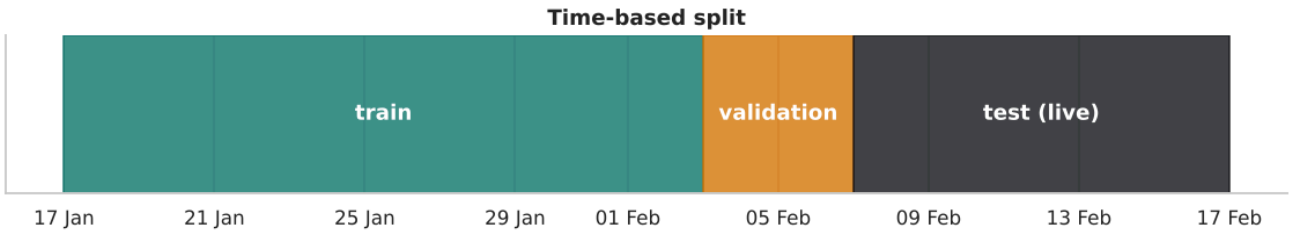
```
Payday rule check: {'2025-01-15': True, '2025-01-31': True, '2025-02-14': True,
'2025-02-15': False, '2025-03-14': True, '2025-03-31': True}
Feature matrix: 96,768 rows × 9 features ['hour', 'dow', 'weekend', 'payday',
'payday_eve', 'bank_i', 'channel_i', 'platform_i', 'region_i']
```

Sample weights: a window with 400 payments is far more reliable than one with 5. The latency model weights each window by √volume and the success model by volume, so quiet night-time noise does not drag the fit.

Step 5 — Train / validation / test split

Time-series data must be split **by time**: train on the past, test on the future.

Set	Dates	Used for
Train	17 Jan – 2 Feb 2025 (17 days, includes the 31 Jan payday)	Fitting models
Validation	3 – 6 Feb 2025 (4 days)	Comparing models and tuning hyperparameters
Test	7 – 16 Feb 2025, the live period, **normal rows only**	Final, untouched evaluation of how well the model knows *normal*
(Production)	All 3 weeks	The final model is re-fitted on train + validation, as the app does




```
train 78,336 windows | validation 18,432 | test 36,235 normal live windows (of
46,044; 9,809 touched by planted events are excluded)
```

split	R^2 (log p95)
random 80/20 (shuffled)	0.829
time-based (train past → test on the future live days)	0.808

The random split reports $R^2 = 0.829$; the honest time-based test gives 0.808. The random split looks better because neighbouring 15-minute windows of the same day end up on both sides, so the model is partly tested on days it has already seen. **Always split time-series data by time.**

Step 6 — Model training & evaluation

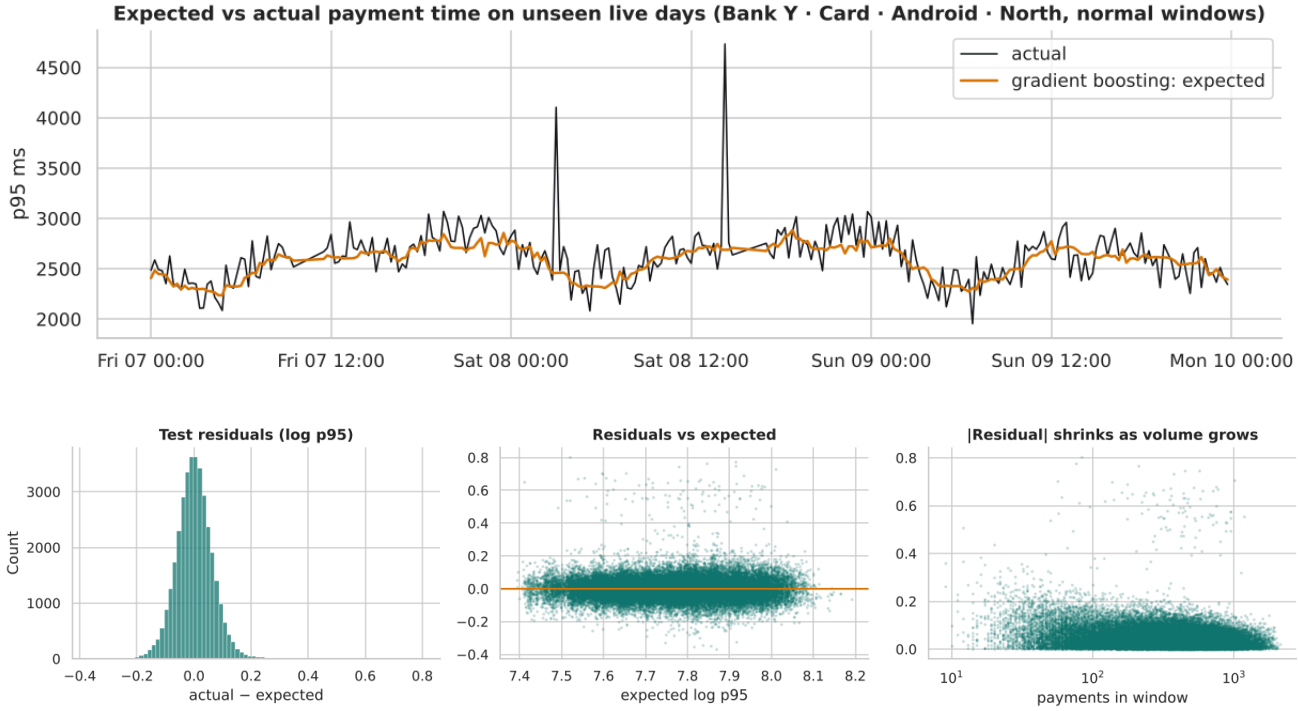
Three candidate models for each target:

1. **Seasonal baseline:** the average for that segment, hour and day type in the training data (what a good spreadsheet would do)
2. **Ridge linear regression:** one-hot features, with hour as a category
3. **Gradient boosting (HistGradientBoosting):** Lumio's production model, with the app's exact settings

target	model	train R^2	validation R^2	validation MAE	test R^2	test MAE	test RMSE
log p95 (payment time)	Seasonal baseline	0.7831	0.7764	0.0547	0.7912	0.0528	0.0729
	Ridge regression	0.7805	0.7706	0.0550	0.7630	0.0569	0.0777
	Gradient boosting	0.8443	0.7961	0.0512	0.8075	0.0500	0.0700
success rate	Seasonal baseline	0.0659	0.0124	0.0070	-0.0010	0.0067	0.0093
	Ridge regression	0.0307	0.0241	0.0070	0.0265	0.0066	0.0092
	Gradient boosting	0.0379	0.0296	0.0070	0.0338	0.0066	0.0091
log volume	Seasonal baseline	0.9459	0.9560	0.1395	0.9381	0.1579	0.2198
	Ridge regression	0.8148	0.8893	0.2265	0.7743	0.2969	0.4198
	Gradient boosting	0.9897	0.9805	0.0852	0.9850	0.0766	0.1082

Why is R^2 so low for success rate? Success rate barely varies over time (about 98 % all day). What variation there is, is mostly **binomial sampling noise**: with a few hundred payments per window, the success rate wobbles by ±0.73 points by pure chance, and no model can predict chance. The model's MAE (0.66 points) is close to that noise floor, so it is doing as well as possible. **R^2 is the wrong yardstick for a nearly constant target**; what matters is that the expected level is right, so that a *drop* stands out (Step 8).

On the untouched live test days, the gradient-boosting latency model is off by **122 ms on average (MAPE 5.0 %)**. Best model on validation for payment time: **Gradient boosting**. Note the train vs validation R^2 gap is small, so **no serious over-fitting**.



Residuals are centred on zero with no pattern against the expected value, but their **size depends on volume**. That's why Lumio does not use one fixed error band: it scales each residual by a **volume-based noise model** (Step 8).

Step 7 — Hyperparameter tuning

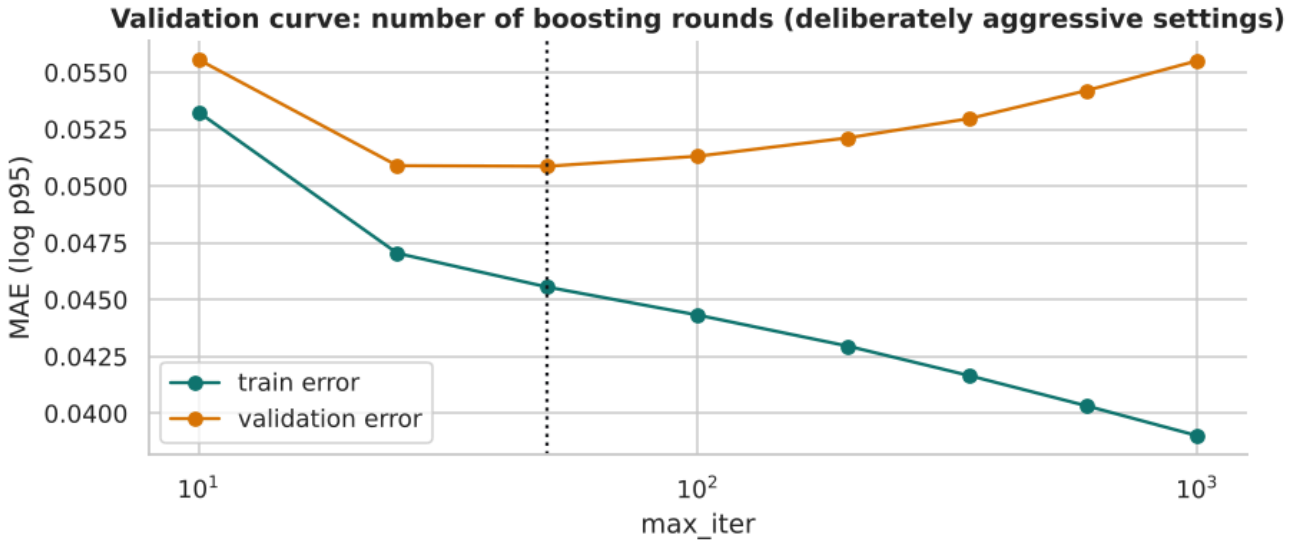
Randomised search over the gradient-boosting settings, scored by **time-series cross-validation** on the training period (each fold trains on the past and validates on the next block). Then the best settings are checked on the validation week.

	params	cv_MAE	train_MAE	std_test_score	mean_fit_time
0	{'min_samples_leaf': 50, 'max_leaf_nodes': 16,...	0.05216	0.04161	0.00731	0.72725
1	{'min_samples_leaf': 100, 'max_leaf_nodes': 16...	0.05233	0.04098	0.00729	1.06354
2	{'min_samples_leaf': 50, 'max_leaf_nodes': 48,...	0.05239	0.04200	0.00749	1.26053
3	{'min_samples_leaf': 50, 'max_leaf_nodes': 16,...	0.05241	0.04059	0.00726	1.14025
4	{'min_samples_leaf': 50, 'max_leaf_nodes': 31,...	0.05248	0.04278	0.00753	0.99226
5	{'min_samples_leaf': 100, 'max_leaf_nodes': 16...	0.05256	0.04063	0.00721	1.19768

```
search: 14 candidates × 3 folds in 37s
```

settings	params	train MAE	validation MAE	test MAE
sklearn defaults	(defaults)	0.04648	0.05065	0.04933
Lumio app settings	{'learning_rate': 0.08, 'max_iter': 350, 'max_...	0.04448	0.05120	0.05004
tuned (best CV)	{'min_samples_leaf': 50, 'max_leaf_nodes': 16,...	0.04655	0.05058	0.04928

Tuning improves validation error by **0.1 %** over the defaults. The app's settings are within **+1.2 %** of the tuned optimum, so they are a sound choice. Small differences like this are typical: **features and data quality matter far more than hyperparameters.**



Train error keeps falling as the model gets more complex, while validation error bottoms out around **50 rounds** and then rises: that rise is **over-fitting**. This is how the number of rounds is chosen.

Step 8 — From prediction to anomaly score

A prediction alone does not raise an alarm. Lumio converts *actual vs expected* into a **z-score**: how many "normal spreads" away the segment is.

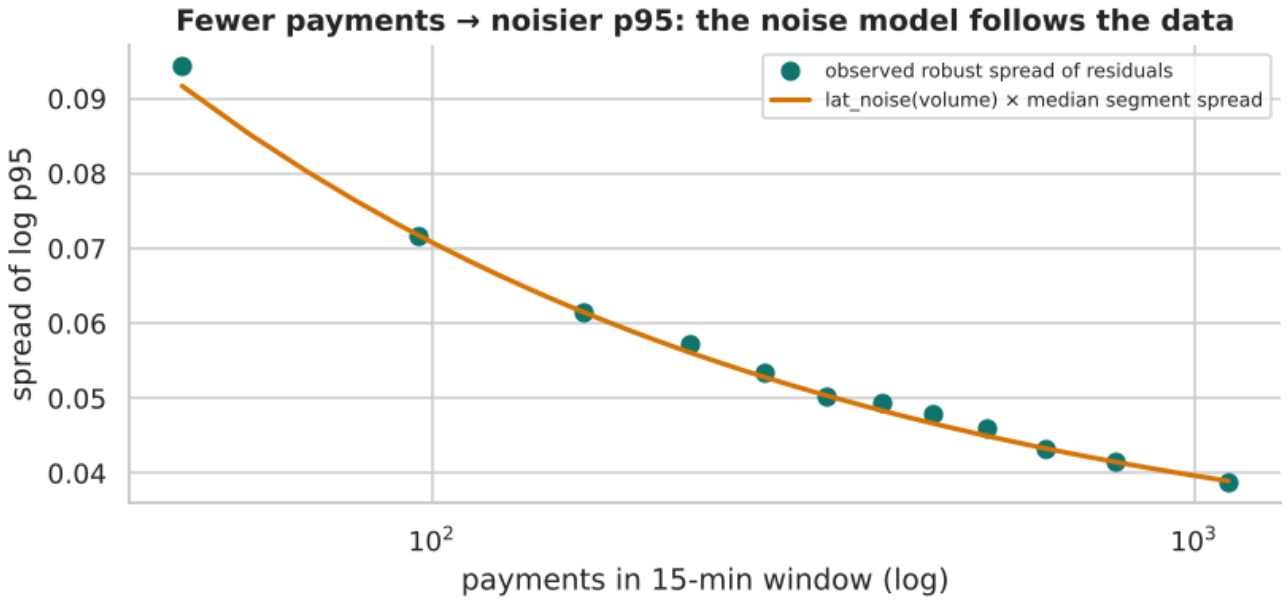
$$z_{\text{latency}} = \frac{\log(\text{actual p95}) - \log(\text{expected p95})}{\text{noise(volume)} \times \text{segment spread}}$$

- **noise(volume)**: sampling noise; fewer payments → larger noise (`lat_noise` , `succ_noise` , `vol_noise`)
- **segment spread**: each segment's own extra variability, estimated **robustly** with the MAD (median absolute deviation × 1.4826), so past outliers don't inflate it

Now we train the model with the **app's own `Baseline().train()`**. It reads the training data from the notebook database with SQL, exactly as in production, and we check that it matches what we built by hand.

```
Baseline.train(): 96,768 windows, 3 models trained in 7.7s
Notebook model == app model (identical predictions on the live data): True
```

	mean	min	50%	max
latency	1.023	0.931	1.012	1.125
success	0.880	0.800	0.861	1.148
volume	0.888	0.829	0.888	0.965



ts	volume	p95_ms	exp_p95	z_lat	success	exp_succ	z_succ	exp_vol	z_vol
2025-02-07 17:00:00	1410	1963.832	2008.255	-0.571	0.981	0.983	-0.464	1333.315	1.026
2025-02-07 19:00:00	1945	4682.220	2133.223	21.100	0.972	0.981	-2.528	1874.703	0.730
2025-02-07 19:30:00	2025	5930.444	2178.569	27.035	0.965	0.981	-4.590	1844.431	1.845
2025-02-07 20:00:00	1947	5669.749	2102.361	26.632	0.959	0.982	-6.133	1750.256	2.081

At 17:00 everything is near z = 0. After the v5.2 rollout (18:58) the latency z-score jumps to **27.0**, far beyond the app's default sensitivity of **4.5**.

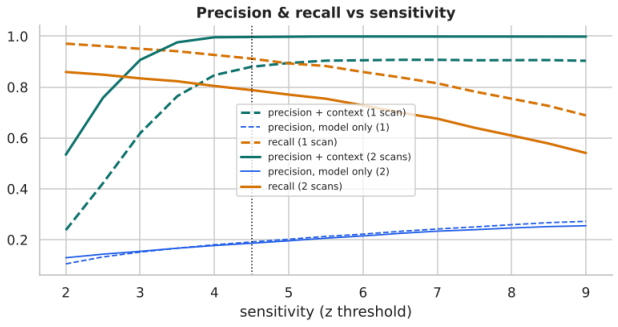
Step 9 — Threshold tuning & detection metrics

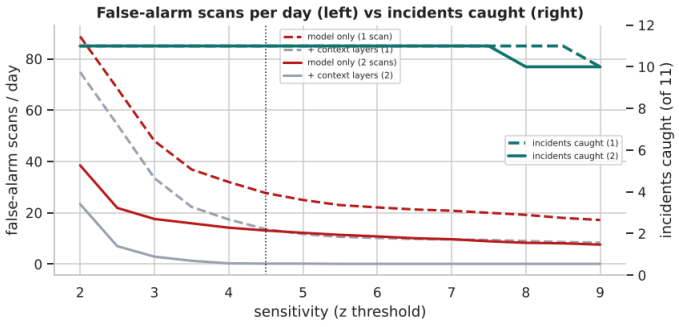
The model gives z-scores; a **threshold** (the *sensitivity* slider in Settings → Alert rules) turns them into flags. The app's rules for each segment window:

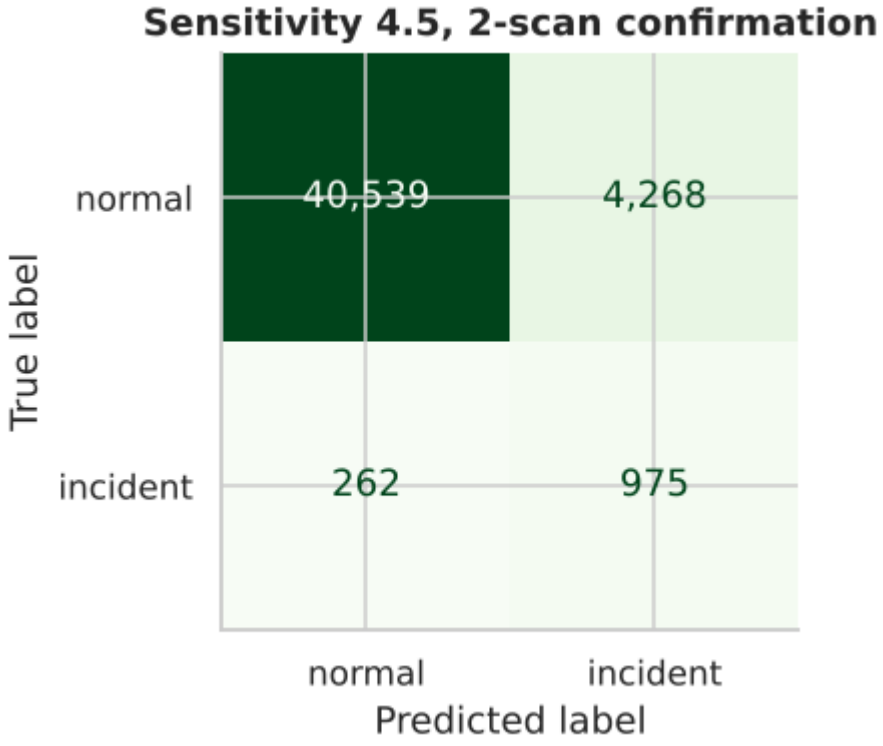
- latency flag: `z_lat > sensitivity` and ≥ 30 payments
- success flag: `z_succ < −sensitivity` and ≥ 30 payments
- volume flag: `|z_vol| > volume_sensitivity` and ≥ 20 expected payments
- **confirmation:** flagged in **2 consecutive scans** before it counts

Evaluated at the **segment-window** level against the answer key (label = inside a real incident's scope and time). Data-quality holds are applied first.

confirm	sensitivity	precision (model only)	precision (+ context layers)	recall	incidents caught	median min to detect	false-alarm scans / day (model only)	false-alarm scans / day (+ context layers)
1 scan (no confirmation)	4.5	0.191	0.880	0.912	11	15.0	27.7	13.6
2 scans (Lumio)	4.5	0.186	0.997	0.788	11	30.0	13.1	0.2







signal	latency	volume	total
where the false alarm happened			
planned maintenance	4	41	45
system-wide demand (storm)	17	346	363
total	21	387	408

At the app's default (sensitivity 4.5, 2-scan confirmation): 11/11 incidents caught, median **30 min** to detect.

Where do the 'false alarms' come from? The table answers it: almost all are **volume** flags during the **winter storm** (real NAB data: demand collapsed everywhere) and **latency** flags during **planned maintenance**. Outside known events there are **0**. That's why the model is only one layer:

- the **system-wide demand advisory** turns the storm's hundreds of segment flags into one advisory to Business Operations, and
- the **change calendar** marks maintenance windows as expected.

With those two context layers, precision rises from **0.19** to **0.997**. Grouping 48 segments into one story and the business guardrails complete the job, which is how the full app scores **19/19 with zero false alarms** (Proof tab).

- **Confirmation** (2 scans) costs about 15 minutes of detection time but halves the model-only false-alarm scans (compare the two rows above).
- **Choosing the sensitivity:** every incident is still caught up to a sensitivity of **7.5**; above that, real incidents start to be missed. The default 4.5 keeps a safety margin for incidents milder than the planted ones. **Recall** at segment-window level is below 1 even when every incident is caught, because the first minutes and the less-affected segments inside a broad scope don't cross the line. The operational measure is the **incident catch rate**.

Step 10 — Live scoring: rebuilding what the dashboard shows

Now we play part of the live period **through the real pipeline**: every 5 minutes a batch is **ingested** into `metrics` (`ingest.store`), **scored** against the model's expectations, and the scores are written to the **`bucket_scores`** table. That table is what the dashboard's main chart reads (`/api/series`). Every 15 minutes the window z-scores (`window_scores`) feed the heatmap and the detector.

We stream **Friday 7 Feb, 08:00 → 23:55**: the card-network outage (10:20) and the app v5.2 release (19:00).

	batch	rows ingested	rows scored	max latency vs expected
0	Fri 08:00	48	48	1.19×
1	Fri 08:05	48	48	1.27×
2	Fri 08:10	48	48	1.15×
3	Fri 08:15	48	48	1.24×
4	Fri 10:20	48	48	3.00×
5	Fri 10:25	48	48	6.42×
6	Fri 19:30	54	48	2.60×

```
288 batches streamed in 11.7s
```

The data changed; the model did not. Expected p95 for Bank Y · Card · Android · North at 10:30, before streaming: 2542 ms; after streaming 288 batches: 2542 ms. New data is *compared with* the model, not *learned into* it. That's what keeps an outage from becoming the new normal.



scan	payments (15 min)	success %	segments scored	held by quality gate	segments over sensitivity 4.5
Fri 09:45	24147	98.18	48	0	0
Fri 10:30	22818	96.36	48	0	24
Fri 19:45	36006	97.86	48	0	3

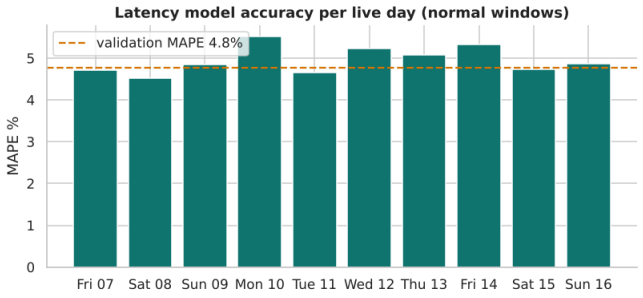
Normal (09:45): all quiet. **Card-network outage (10:30):** every *Card* row lights up across all banks, regions and platforms. The detector's grouping step turns those 24 segments into **one** alert, 'channel = Card', and the investigation finds the card-network service. **Release (19:45):** only the Bank X · UPI · Android row lights up, which points straight at something specific to that bank's Android app: the v5.2 rollout.

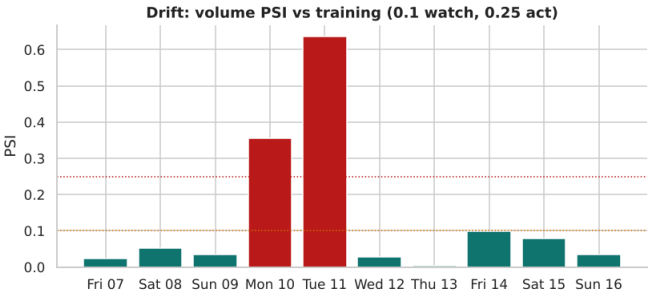
Dashboard ↔ pipeline map

What you see on the dashboard	Pipeline step	Where it lives
Live chart: actual, **expected**, grey band, **amber alert line**	Steps 6–8 (model + noise) and 10 (scoring)	`bucket_scores`
Heatmap of 48 segments	Step 8 z-scores per 15-min window	`window_scores`
KPI strip (payments, success %, segments abnormal)	Step 10 scan	`scans`
Data quality tab	Step 2 quality gate	`dq_checks`
Alert cards, *Incidents* tab, e-mails	Step 9 rules + grouping + confirmation, then investigation	`incidents` , `outbox`
Settings → *sensitivity* slider	Step 9 threshold	`settings`
Proof tab	Step 9 evaluation against the answer key	`ground_truth`
Learning tab ("not useful" → threshold +1)	Feedback loop (per-segment threshold, not retraining)	`feedback` , `sensitivity`

Step 11 — Monitoring & retraining

A model in production must be **watched**. Two questions: *is it still accurate?* and *is the data still like the training data?* (**drift**)





day	Fri 07	Sat 08	Sun 09	Mon 10	Tue 11	Wed 12	Thu 13	Fri 14	Sat 15	Sun 16
MAPE	4.708	4.521	4.839	5.516	4.664	5.238	5.074	5.329	4.739	4.874
windows	4345.000	4392.000	4272.000	2286.000	96.000	4260.000	4332.000	3264.000	4524.000	4464.000
volume PSI	0.023	0.052	0.035	0.356	0.636	0.027	0.004	0.098	0.079	0.035

Accuracy stays close to the validation level every day, so **no retraining is needed** during these ten days. The drift check flags **Mon 10 and Tue 11** (PSI up to 0.64): the winter storm. That is a **temporary** shift, not a new normal, so the right response is an *advisory*, not retraining. Retraining on storm days would teach the model that empty streets are normal.

Retraining policy (production recommendation)

Trigger	Action
Scheduled: monthly	Re-fit on a rolling 8–12 weeks of history, **excluding** windows inside confirmed incidents
Accuracy: daily MAPE above 1.5× validation for 5+ days	Investigate, then retrain
Drift: PSI > 0.25 for 7+ days (a *lasting* change, e.g. a new bank onboarded)	Retrain
Structural change: new segment, app, region	Retrain (new categories)
Feedback: many "not useful" on one segment	Adjust that segment's threshold first (the *Learning* tab); retrain only if it persists

Every new model is **compared with the current one on the same validation week** before it replaces it, the same champion / challenger idea as Steps 6–7.

```
Saved baseline.joblib (3.8 MB). Reloaded model gives identical predictions: True
In the app, Detector() loads models/baseline.joblib at start-up, so training
happens once, not on every run.
```

Step 12 — Summary

Pipeline step	What we did	Key result
Load	CSV → SQL (Lumio schema), reconciliation, streaming ingest	CSV and database match exactly; one 5-min batch = 48 rows summarising ~6,642 payments
Clean	Validation, quality gate, mixture-p95 aggregation	Missing collector → *hold*; duplicates → *dedup*; no false pages from bad data
EDA	Distributions, seasonality, payday, noise vs volume	Log transform for time and volume; payday evening 1.7× a normal Friday
Features	9 calendar + segment features; nothing measured "now"	Avoids leakage; expectations can be pre-computed
Split	Time-based train / validation / test	Random split R^2 0.829 vs honest time split 0.808
Train & evaluate	Seasonal baseline vs Ridge vs gradient boosting	Gradient-boosting latency test R^2 **0.807**, error ≈ **122 ms** (MAPE 5.0 %)
Tune	Randomised search + time-series CV; validation curve	Tuned validation MAE 0.0506 vs app 0.0512 vs defaults 0.0507
Score	Noise model × robust segment spread → z	Notebook model reproduces the app's model exactly
Threshold	Sensitivity sweep, confirmation, false-alarm breakdown	At 4.5 + 2 scans: 11/11 incidents, median 30 min to detect; precision 0.19 model-only → 1.00 with storm advisory + change calendar
Live	Stream → ingest → score → `bucket_scores` → dashboard chart + heatmap	The data updates every 5 min; the model stays fixed
Monitor	Daily accuracy, PSI drift, save/load	Stable accuracy; storm detected as temporary drift → advisory, not retraining

---

Professor's likely questions on the pipeline

1. Does the model update with every transaction? No. Transactions are summarised into 5-minute rows; the model scores each new row against what it learned from history. It is re-trained on a schedule, not continuously, so outages never become "normal" (Step 0, Step 10).

2. Why aggregate to 5 / 15 minutes instead of scoring each payment? A single payment carries almost no signal; a few hundred have a measurable average, failure rate and p95 with known noise. Aggregation also scales to millions of payments and protects privacy.

3. How did you split the data, and why not randomly? By time: train 17 Jan – 2 Feb, validate 3 – 6 Feb, test on the 10 live days. A random split leaks neighbouring windows from the same day into training and overstates accuracy (Step 5).

4. What features did you engineer, and what did you leave out on purpose? Calendar (fractional hour, weekday, weekend, payday, payday evening) and segment identity. Anything measured "now" was left out, because it would leak the anomaly into the expectation (Step 4).

5. Why log-transform the targets? Payment time and volume are right-skewed and change multiplicatively; logs make residuals symmetric and comparable across segments (Step 3).

6. Why gradient boosting rather than linear regression? It captures interactions (e.g. Card × evening × payday) without hand-made terms, and it was best or equal on validation. Ridge and the seasonal baseline were kept as benchmarks (Step 6).

7. How did you tune it, and did tuning matter much? Randomised search with time-series CV, then checked on validation; the validation curve shows where over-fitting starts. The gain was modest: features and data quality mattered more (Step 7).

8. How do you avoid over-fitting? Time-based validation, the train vs validation gap, regularisation (min samples per leaf, L2), and a validation curve on boosting rounds.

9. Which evaluation metrics, and why not accuracy? MAE, RMSE, R^2, MAPE for the regression; precision, recall, F1, incidents caught and time to detect for detection. Incidents are rare, so accuracy would be misleading (Steps 6, 9).

10. How was the alert threshold chosen? A sweep of sensitivity against precision, recall, false alarms and incidents caught, plus 2-scan confirmation. The default keeps every incident caught with a safety margin; the false alarms that remain come from known contexts (storm, maintenance), which later layers handle (Step 9).

10b. Your model-only precision is low. Isn't that a weak model? No. The breakdown shows almost all of those flags are real volume collapses during the storm and real slowness during planned maintenance. The *data* really was abnormal, but it was not an incident. Separating "abnormal" from "actionable" is the job of the context layers, and with them precision is close to 1.

11. Why does a small segment get a wider band? Fewer payments mean more sampling noise. The z-score divides by a volume-based noise model times a robust (MAD) per-segment spread (Steps 3, 8).

12. What happens when data is missing or duplicated? The quality gate holds the segment (no scoring, a data ticket) or removes duplicates before scoring (Step 2).

13. How would you know when to retrain? Monitor daily error against the validation level and drift (PSI); retrain on schedule or on a lasting shift, never on a temporary event like the storm

(Step 11).

14. Is the notebook really the same as the app? Yes. It imports the app's modules, trains with `Baseline().train()` from the same SQL table, and shows the predictions are identical (Step 8).